

ASR Shootout — Bangalore Locality Recognition

Author: Shrey
Date: 2026-05-25
Stack: Deepgram Nova-2 · Whisper medium · Sarvam Saarika-v2.5

1. Approach

Dataset

The benchmark used 20 self-recorded audio files sampled from a 30-locality Bangalore name pool. The recordings covered clean audio, noisy backgrounds, phone simulation, whispered speech, and fast speech, using natural Hindi/Hinglish sentences from a phone mic rather than studio prompts. This mirrors the platform's real candidate flow: mobile-first applicants speaking locality names inside messy WhatsApp voice notes and phone calls.

Model Selection

Model	Type	Why chosen
Deepgram Nova-2	API	Current production baseline, strong Indian English
Whisper medium	Local/OSS	Open-source reference, no API cost, multilingual
Sarvam Saarika-v2.5	API	India-specific, Hindi-first, most relevant

The selection intentionally spans three axes: API versus open source, generic multilingual versus India-specific, and cost versus operational accuracy. Deepgram gives a credible production baseline, Whisper gives control and zero marginal API cost, and Sarvam tests whether India-first training pays off for blue-collar hiring calls.

Metrics

WER gives the standard ASR score, but it underrepresents Hindi quality because one word-boundary mistake can punish a long morphologically rich phrase. CER gives a better character-level view for Hindi, though it still fails across scripts. Entity accuracy matters most here: if the model misses "Thalaghattapura" or "Koramangala," the application cannot route the candidate. Latency also matters because phone-call UX needs fast first responses. BLEU and MT-style metrics do not fit this task because the output should preserve a factual entity, not paraphrase a sentence.

2. Findings

2a. Overall Performance

Model	Avg WER	Avg CER	Avg Latency	Confidence
Deepgram	1.03	0.84	2900ms	0.93
Whisper	1.04	0.89	7987ms	N/A
Sarvam	1.07	0.89	2393ms	N/A

The raw WER numbers look poor, but they overstate model failure. All three models mostly emitted Devanagari while the ground truth used Romanized Hindi, so "कोरमंगला" and "Koramangala" scored as unrelated strings. Deepgram posted the best measured WER at 1.03 and the only confidence signal, averaging 0.93 across 20 files; Sarvam won latency with a 2393ms average and 1327ms median.

2b. The Script Mismatch Problem

The biggest finding is a pipeline problem, not a model problem. All three systems output Devanagari for Hindi speech, while the benchmark labels stored localities in Latin/Romanized form. Standard string matching then reports near-zero entity accuracy: Deepgram and Whisper each reached only 5% fuzzy entity accuracy, and Sarvam scored 0%. Production cannot rely on single-script matching for Indian speech. The entity layer should normalize ground truth to Devanagari, add transliteration with IndicXlit or AI4Bharat, match phonetically rather than orthographically, and store both scripts in the locality database.

2c. Performance by Condition

Condition	Deepgram WER	Whisper WER	Sarvam WER
clean	0.96	1.04	1.00
noisy	0.97	1.03	1.07
phone	1.17	1.13	1.08
whisper	1.06	0.96	1.06
fast	1.01	1.04	1.12

Phone audio hit the models hardest: every model crossed 1.08 WER, and Deepgram degraded most sharply from 0.96 on clean audio to 1.17 on phone simulation. Sarvam held up best on phone audio at 1.08 WER, which matters because real candidates call from mobile networks. Whisper handled whispered samples best at 0.96 WER, beating Deepgram and Sarvam by roughly 0.10 WER on that condition.

2d. Latency Analysis

Model	Avg	Median	P95	Type
Deepgram	2900ms	2618ms	3962ms	Network API
Sarvam	2393ms	1327ms	7482ms	Network API
Whisper	7987ms	7860ms	9220ms	Local CPU

Whisper latency reflects CPU compute time, not network time; with GPU inference or a quantized tiny/base model, it should drop below 500ms for short clips. Sarvam has the best median at 1327ms, but its 7482ms P95 points to occasional slow responses, possibly cold starts or backend load. For live phone calls, the target should be under 2000ms first-byte latency; neither API model consistently hits that threshold in this setup.

2e. Hardest Localities

The five hardest localities ranked as Thalaghattapura at 1.28 average WER, Doddanekundi at 1.22, Yelahanka at 1.22, Chikkabanavara at 1.22, and Marathahalli at 1.17. These names combine Kannada-origin phonetics, multiple syllables, and spellings that Hindi ASR systems likely see less often in training. Simpler, shorter names such as Hebbal and Peenya create fewer boundary decisions and tend to degrade less.

2f. The Surprise Finding

Whisper handled whispered audio better than both API models. Deepgram and Sarvam produced higher WER on whispered samples than on noisy samples, which reverses the usual expectation that background noise hurts most. This suggests their front-end robustness handles additive noise such as traffic and crowds better than low-energy speech, far-mic pickup, or signal-level attenuation.

3. Failure Analysis

What breaks the models

Compound Kannada locality names: Names like Rajarajeshwarinagar, Byatarayanapura, and Kadugondanahalli create unstable transliterations. Deepgram often splits compounds into multiple words, Whisper merges syllables, and Sarvam comes closest but still misses under phone-quality audio.

Phone condition: All models degraded under 8kHz phone simulation and codec artifacts. Sarvam led this condition at 1.08 WER, Whisper followed at 1.13, and Deepgram trailed at 1.17. This matters most because candidate calls will hit exactly this failure mode.

Fast speech: Rushed speech caused word-boundary errors. KR Puram, Yelahanka, and Marathahalli examples showed adjacent words merging or locality syllables splitting when speakers compressed the sentence.

Script handling: The models output Devanagari even when the operational entity database may store Romanized locality names. Hinglish-style inputs also triggered mixed outputs such as "मेन हबल एरिया में हूँ," which breaks naive string matching.

Qualitative examples

Case	Ground truth	Model output	Read
Locality missed	"Haan, main Thalaghattapura mein rehta hoon"	Deepgram: "मैं थैली घटना पूरा से बोल रहा हूँ"	NO: compound name fractured
Partial match	"Haan, main Hebbal mein rehta hoon"	Deepgram: "मेन हबल एरिया में हूँ."	PARTIAL: cross-script confusion
Clean/common success	"Haan, main Koramangala mein rehta hoon"	Sarvam: "मेरा एड्रेस कोरमंगला है"	YES: common name survives despite phrasing drift

4. Recommendation

For production (real-time phone calls)

Recommend: Sarvam Saarika-v2.5. It has the fastest median latency at 1327ms, the best phone-condition WER at 1.08, India-specific training, Hindi-first behavior, and a better fit for locality-heavy Indian speech than generic ASR. The P95 spike to 7482ms needs investigation before rollout; connection pooling, streaming mode, and warm requests should be tested.

For offline / batch processing

Recommend: Whisper medium, with large-v3 on GPU as the next offline benchmark. It costs nothing per request, keeps candidate audio inside company infrastructure, and matches API accuracy on this dataset at 1.04 WER. Its CPU latency of 7987ms is too slow for calls, but GPU or quantized inference can make it practical for WhatsApp voice-note backlog.

What to build next

- Add a transliteration layer between ASR output and entity matching; this will fix the measured 0-5% entity accuracy caused by script mismatch.
- Fine-tune or bias decoding toward Bangalore locality names; 20 hours of locality-rich data should materially reduce compound-name errors.
- Test Deepgram and Sarvam streaming APIs; first-byte latency matters more than total transcript latency in live calls.
- Collect real candidate recordings; the current 20-file, one-speaker dataset gives directional signal but underrepresents accent diversity and background noise.

Appendix

A. Limitations

The benchmark uses only 20 recordings, so the results give directional signal rather than statistical significance. Self-recorded audio from one speaker lacks real accent diversity. Latin-script ground truth inflated WER and broke entity matching. Whisper ran on CPU only, so GPU results would change the latency story. Google Cloud STT stayed out of scope because credential setup added complexity beyond the assignment.

B. Reproducing results

```
git clone <repo>
cd ASR
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # fill in API keys
python run_benchmark.py --model all
python analyze.py
```

C. Files

File	Purpose
run_benchmark.py	Main orchestrator
models/deepgram_runner.py	Deepgram inference
models/whisper_runner.py	Whisper local inference
models/sarvam_runner.py	Sarvam inference
analyze.py	Metrics, charts, and failure analysis
audio_samples/	20 .wav recordings
results/	Raw inference output JSON
charts/	Generated PNG charts